

Searching Algorithms

Table of Contents

Introduction

Linear Search

Binary Search

Jump Search

Interpolation

Search

Searching Algorithms

- Searching Algorithm is an algorithm made up of a series of instructions that retrieves information stored within some data structure, or calculated in the search space of a problem domain.
- There are many sorting algorithms, such as:
 - Linear Search, Binary Search, Jump Search, Interpolation Search, Ternary Search

Linear Search

Linear Search

Linear Search is a method for finding a target value within a list. It **sequentially** checks each element of the list for the target value until a match is found or until all the elements have been searched.

Linear Search

❑ Algorithm:

- Step1: Start from the leftmost element of array and one by one compare the element/value(x) with each element of array.
- Step2: If x matches with an element, return the index.
- Step3: If x doesn't match with any of elements, return -1.

Linear Search

❑ Assume the following Array:

❑ Search for 9

8	12	5	9	2
----------	-----------	----------	----------	----------

Linear Search

☐ Compare

☐ X=

9

8

12

5

9

2

☐

i

Linear Search

☐ Compare

☐ X=

9

8	12	5	9	2
---	----	---	---	---

☐

i

Linear Search

☐ Compare

☐ X =

9

8	12	5	9	2
---	----	---	---	---

☐

i

Linear Search

❑ Compare

❑ X=

9

8	12	5	9	2
---	----	---	---	---

❑
i

Linear Search

❑ Found at index = 3

8	12	5	9	2
---	----	---	---	---

Linear Search

❏ Code

```
// Linearly search x in arr[]. If x is present then return its location,  
//otherwise return -1
```

```
int search(int arr[], int n, int x)  
{  
    for (int i = 0; i < n; i++)  
        if (arr[i] == x)  
            return i;  
    return -1;  
}
```

Linear Search

- ❑ Time Complexity: $O(n)$

- ❑ Example of worst case: search for the last element

4	6	8	9	1
---	---	---	---	---

Binary Search

Binary Search

❑ Binary Search is the most popular Search algorithm.

It is efficient and also one of the most commonly used techniques that is used to solve problems.

Binary search uses **sorted array** by repeatedly dividing the search interval in half.

Binary Search

❑ Algorithm:

- Step1: Compare x with the middle element.
- Step2: If x matches with middle element, we return the mid index.
- Step3: Else If x is greater than the mid element, search on right half.
- Step4: Else If x is smaller than the mid element. search on left half.

Binary Search

❑ Assume the following Array:

❑ Search for 40

2

3

1

3

4

5

7

0

0

0

0

0

Binary Search

☐ Compare

☐ X = 40

2	3	1	3	4	5	7
☐		0	0	0	0	0
L			mid			R

Binary Search

☐ Compare

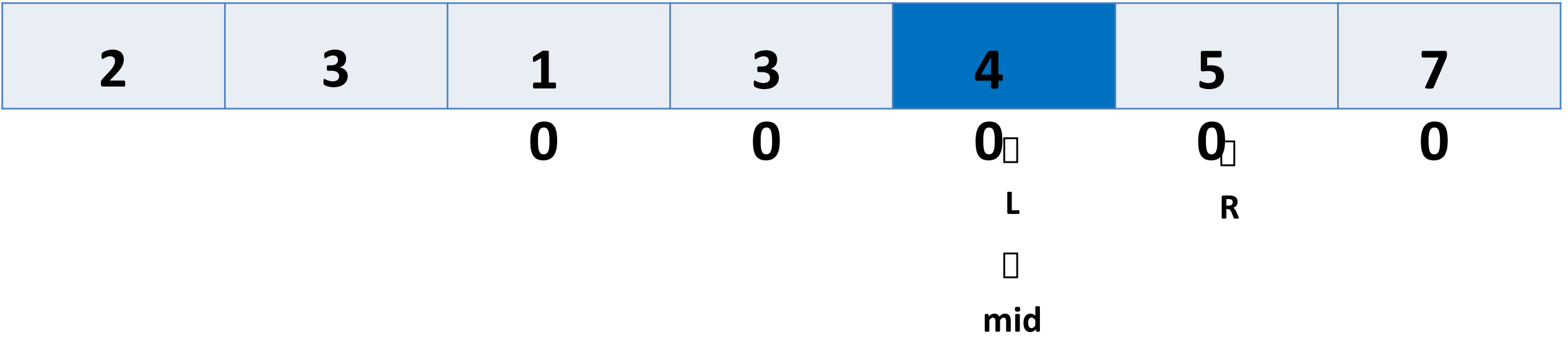
☐ X = 40

2	3	1	3	4	5	7
		0	0	0	0	0
				L	mid	R

Binary Search

☐ Compare

☐ X = 40



Binary Search

☐ x=40 , found at index = 4

2	3	1	3	4	5	7
		0	0	0	0	0

Binary Search

❑ C++ Implementation:

// A **recursive binary** search function. It returns location of x in given array arr[l..r] is present, otherwise -1

```
int binarySearch(int arr[], int l, int r, int x)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;
        // If the element is present at the middle itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }
    return -1; }
```

Binary Search

Time Complexity: $O(\log_2 n)$

Jump Search

Jump Search

□ Jump Search is a searching algorithm for **sorted arrays**. The basic idea is to check fewer elements (than linear search) by jumping ahead by fixed steps or skipping some elements in place of searching all elements.

Jump Search

❑ Algorithm:

- Step1: Calculate Jump size
- Step2: Jump from index i to index $i + \text{jump}$
- Step3: If $x == \text{arr}[i + \text{jump}]$ return x
- Else jump back a step
- Step4: Perform linear search

Jump Search

❑ Assume the following sorted array:

❑ Search for 77

0	1	1	2	3	5	8	13	21	34	55	77	89	91	95	110
---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	-----

Jump Search

❑ Calculate:

- Size of array $n = 16$
- Jump size = $\text{sqrt}(n) = 4$

0	1	1	2	3	5	8	13	21	34	55	77	89	91	95	110
---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	-----

Jump Search

❑ Jump size = 4

❑ Search from index 0

❑ Compare index value with search number $0 < 77$

77

0

1

1

2

3

5

8

13

21

34

55

77

89

91

95

110

Jump Search

❑ Jump size = 4

❑ Jump from index 0 to index 3

❑ Compare index value with search number $2 < 77$

77

0

1

1

2

3

5

8

13

21

34

55

77

89

91

95

110

Jump Search

- ❑ Jump size = 4

- ❑ Jump from index 3 to index 6

- ❑ Compare index value with search number $8 < 77$

77

0

1

1

2

3

5

8

13

21

34

55

77

89

91

95

110

Jump Search

- ❑ Jump size = 4
- ❑ Jump from index 6 to index 9
- ❑ Compare index value with search number $34 < 77$

77

0

1

1

2

3

5

8

13

21

34

55

77

89

91

95

110

Jump Search

- ❑ Jump size = 4
- ❑ Jump from index 9 to index 12
- ❑ Compare index value with search number $89 > 77$

77

0	1	1	2	3	5	8	13	21	34	55	77	89	91	95	110
---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	-----

Jump Search

- ❑ jump back a step
- ❑ Perform linear search
- ❑ Compare found at index 11

77

0

1

1

2

3

5

8

13

21

34

55

77

89

91

95

110

Jump Search

❑ C++ Code

```
int jumpSearch(int arr[], int x, int n)
{
    // Finding block size to be jumped
    int step = sqrt(n);

    // Finding the block where element is
    // present (if it is present)
    int prev = 0;
    while (arr[min(step, n)-1] < x)
    {
        prev = step;
        step += sqrt(n);
        if (prev >= n)
            return -1;
    }

    // Doing a linear search for x in block
    // beginning with prev.
```

Jump Search

```
while (arr[prev] < x)
{
    prev++;

    // If we reached next block or end of
    // array, element is not present.
    if (prev == min(step, n))
        return -1;
}
// If element is found
if (arr[prev] == x)
    return prev;

return -1;
}
```

Time Complexity: $O(\sqrt{n})$

Interpolation Search

Interpolation Search

- The Interpolation Search is an improvement over Binary Search for instances. On the other hand interpolation search may go to different locations according the value of key being searched.

Interpolation Search

❑ Algorithm:

- Step1: In a loop, calculate the value of “pos” using the position formula.
- Step2: If it is a match, return the index of the item, and exit.
- Step3: If the item is less than $\text{arr}[\text{pos}]$, calculate the position of the left sub-array. Otherwise calculate the same in the right sub-array.
- Step4: Repeat until a match is found or the sub-array reduces to zero.

Interpolation Search

The idea of formula is to return higher value of pos when element to be searched is closer to arr[hi]. And smaller value when closer to arr[lo].

$$\text{pos} = \text{lo} + [(x - \text{arr}[\text{lo}]) * (\text{hi} - \text{lo}) / (\text{arr}[\text{hi}] - \text{arr}[\text{Lo}])]$$

arr[] ==> Array where elements need to be

^{searched}
x ==> Element to be searched

lo ==> Starting index in arr[]

hi ==> Ending index in arr[]

Interpolation Search

❑ Assume the following sorted array:

❑ Search for $x = 18$

10	12	13	16	18	19	20	21	22	23	24	33	35	42	47
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Interpolation Search

❑ Calculate $\text{pos} = \text{lo} + [(x - \text{arr}[\text{lo}]) * (\text{hi} - \text{lo}) / (\text{arr}[\text{hi}] - \text{arr}[\text{Lo}])]$

❑ $\text{Lo}=0$, $\text{hi}=14$, $\text{arr}[\text{lo}]=10$, $\text{arr}[\text{hi}]= 47$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	13	16	18	19	20	21	22	23	24	33	35	42	47

Interpolation Search

- ❑ Calculate pos = 3
- ❑ Compare with x = 18

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	13	16	18	19	20	21	22	23	24	33	35	42	47

Interpolation Search

❑ Calculate $\text{pos} = \text{lo} + [(x - \text{arr}[\text{lo}]) * (\text{hi} - \text{lo}) / (\text{arr}[\text{hi}] - \text{arr}[\text{Lo}])]$

❑ $\text{Lo}=4$, $\text{hi}=14$, $\text{arr}[\text{lo}]=18$, $\text{arr}[\text{hi}]= 47$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	13	16	18	19	20	21	22	23	24	33	35	42	47

Interpolation Search

❑ Calculate pos = 4

❑ Compare with $x=18$, found at index 4

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	13	16	18	19	20	21	22	23	24	33	35	42	47

Interpolation Search

❑ C++ Code

```
int interpolationSearch(int arr[], int n, int x)
{
    // Find indexes of two corners
    int lo = 0, hi = (n - 1);

    // Since array is sorted, an element present
    // in array must be in range defined by corner
    while (lo <= hi && x >= arr[lo] && x <= arr[hi])
    {
        if (lo == hi)
        {
            if (arr[lo] == x) return lo;
            return -1;
        }
        // Probing the position with keeping
        // uniform distribution in mind.
        int pos = lo + (((double)(hi - lo) /
            (arr[hi] - arr[lo])) * (x - arr[lo]));
```

Interpolation Search

```
// Condition of target found
if (arr[pos] == x)
    return pos;

// If x is larger, x is in upper part
if (arr[pos] < x)
    lo = pos + 1;

// If x is smaller, x is in the lower part
else
    hi = pos - 1;
}
return -1;
}
```


Interpolation Search

- Time Complexity: If elements are uniformly distributed,
- then $O(\log(\log n))$. In worst case it can take up to $O(n)$.